Assignment-13.5

Name : G. Dhanush

Ht.no : 2303A51619

Batch. No: 22

Task Description #1 (Refactoring – Removing Global Variables)

Prompt:

Refactor the given Python code to remove the global variable `rate`.

Pass the value as a function to improve modularity and testability and adding clear documentation.

Ensure the function works for different interest rates and take input from user.

rate = 0.1

def calculate_interest(amount):

  return amount * rate

print(calculate_interest(1000))

Code:

```
def calculate_total_cost(price, quantity, rate):
    subtotal = price * quanti (variable) subtotal: Any
    total_cost = subtotal + (subtotal * rate)
    return total_cost

price = float(input("Enter the price of the item: "))
quantity = int(input("Enter the quantity of the item: "))
rate = float(input("Enter the tax rate (as a decimal): "))
total_cost = calculate_total_cost(price, quantity, rate)
print("The total cost is:", total_cost)
```

Output:

```
Enter the price of the item: 650
Enter the quantity of the item: 2
Enter the tax rate (as a decimal): 0.2
The total cost is: 1560.0
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>
```

Observation:

The updated code gets rid of the global variable and instead passes the interest rate as a parameter, making it more modular. This way, the function is easier to test with different values and doesn't rely on outside variables. The structure gets simpler, easier to reuse, and easier to keep up over time.

Task Description #2 : (Refactoring Deeply Nested Conditionals)
Prompt:

Refactor the following Python code that contains deeply nested if–elif–else statements.

Simplify the control flow using guard clauses or a mapping-based approach to improve readability and maintainability.

Remove unnecessary nesting, apply clear variable naming, and improve the logic while keeping the same output format.

And take input from the user for the score.

Legacy Code:

```
score = 78
if score >= 90:
    print("Excellent")
else:
    if score >= 75:
        print("Very Good")
    else:
        if score >= 60:
            print("Good")
        else:
            print("Needs Improvement")
```

Code:

```python
def evaluate_score(score):
    if score >= 90:
        return "Excellent"
    elif score >= 75:
        return "Very Good"
    elif score >= 60:
        return "Good"
    else:
        return "Needs Improvement"

# Take input from user
score = int(input("Enter the score: "))

# Call function
grade = evaluate_score(score)

# Print result
print("The grade for the score", score, "is:", grade)
```

Output:

```
Enter the score: 80
The grade for the score 80 is: Very Good
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>  c:; cd 'c
y\AppData\Local\Programs\Python\Python313\python.ex
y-2025.18.0-win32-x64\bundled\libs\debugpy\launcher

Enter the score: 23
The grade for the score 23 is: Needs Improvement
```

Observation:

The refactored code makes things easier to read by getting rid of those complicated nested ifs and using a simpler control structure instead.Simplifying the logic makes the program easier to follow and keep up, without changing what it actually does.This way, the code stays simpler and it's easier to keep up with any changes down the road.

Task Description #3 : (Refactoring Repeated File Handling Code)
Prompt:

Refactor the repeated file handling code using Python.

Apply DRY principle and context managers (with open()).

Create a reusable function that accepts the filename as a parameter, removes duplicate open/read/close code, improves variable naming,

and adds comments explaining the logic.Take input from the user for the filename.

Legacy Code:

```
f = open("data1.txt")

print(f.read())

f.close()

f = open("data2.txt")

print(f.read())

f.close()
```

Code:

```
C: > Users > Dhanush Reddy > 3RD Year > AIAC > 13.5.py > ...
f.close()
Refactor the given legacy file handling code in Python by applying the DRY principle and using a reusable function with

"""
def read_file(filename):
    """
    Reads the content of a file and returns it as a string.

    Parameters:
    filename (str): The name of the file to be read.

    Returns:
    str: The content of the file.
    """
    try:
        with open(filename, 'r') as file:
            content = file.read()
            return content
    except FileNotFoundError:
        print(f"The file {filename} was not found.")
        return None
    except IOError:
        print(f"An error occurred while reading the file {filename}.")
        return None

# Example usage:
if __name__ == "__main__":
    user_input = input("Enter the filename to read: ")
    file_content = read_file(user_input)
    if file_content is not None:
        print("File content:")
        print(file_content)
```

Output:

```
Enter the filename to read: user_activity.log
File content:
2026-01-25 14:31:44,513 - User: Myst, IP Address: 123456, Timestamp: 2026-01-25 14:31:44
2026-01-29 13:28:25,217 - User: 1432, IP Address: 101.879.872.0, Timestamp: 2026-01-29 13:28:25
```

Observation:

The updated code gets rid of repeating file handling parts by adding a reusable function where you just pass in the file name.Using the with open() context manager helps manage resources better by making sure files get closed automatically when you're done with them.This method sticks to the DRY principle, makes the code easier to follow, and helps keep it simple to maintain.

Task Description #4 : (Optimizing Search Logic)

Prompt:

Refactor the given Python code that performs a linear search on a list of usernames.

Replace the list-based linear search with a more efficient data structure like a set or dictionary to achieve better time complexity.

Include meaningful variable names, comments explaining the changes, and mention the time complexity of the new approach

and Take input from the user for the username.

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

  if u == name:

    found = True

    break

print("Access Granted" if found else "Access Denied")

Code:

```
def check_user_access(username, user_set):
    """
    This function checks if a given username has access by looking it up in a set of authorized usernames.

    Parameters:
    username (str): The username to check for access.
    user_set (set): A set containing the authorized usernames.

    Returns:
    bool: True if the username has access, False otherwise.
    """
    return username in user_set

# create a set of valid usernames for efficient lookup
valid_usernames = {"admin", "guest", "editor", "viewer"}

# take username input from the user
input_username = input("Please enter your username: ")

# check if the input username has access
if check_user_access(input_username, valid_usernames):
    print("Access granted.")
else:
    print("Access denied.")
```

Output:

```
Year\AIAC\13.5.py'
Please enter your username: Dhanush
Access denied.
PS C:\Users\Dhanush Reddy\3RD Year\AIAC> ^C
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>  c:; cd 'c
n.exe' 'c:\Users\Dhanush Reddy\.vscode\extensions\m
Year\AIAC\13.5.py'
Please enter your username: admin
Access granted.
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>
```

Observation:

The updated program makes searching faster by switching from a list to a set, which cuts down the lookup time from O(n) to about O(1).The control flow is easier to follow by checking membership directly instead of using a loop with a flag variable.This makes the code cleaner, easier to read, and better in performance.

Task Description #5 : (Refactoring Procedural Code into OOP Design)
Prompt:

Refactor the given Python code into an Object-Oriented design.

Create a class named EmployeeSalaryCalculator that follows OOP principles and encapsulation.

Include attributes for salary and tax_rate, and methods to calculate tax and net salary.

Adding proper documentation. Ensure the code is clean, modular, and readabile. Take input from the user for salary and tax rate.

Legacy Code:

```
salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)
```

Code:

```python
class EmployeeSalaryCalculator:
    """
    A class to calculate the salary of employees based on their working hours and hourly rate.
    Attributes:
        name (str): The name of the employee.
        hours_worked (float): The number of hours worked by the employee.
        hourly_rate (float): The hourly rate of the employee.
    Methods:
        calculate_salary(): Calculates the total salary of the employee based on hours worked and hourly rate.
        calculate_net_salary(): Calculates the net salary of the employee after
    """

    def __init__(self, salary, tax_rate):
        self.salary = salary
        self.tax_rate = tax_rate

    def calculate_salary(self):
        return self.salary

    def calculate_net_salary(self):
        tax_amount = self.salary * self.tax_rate
        net_salary = self.salary - tax_amount
        return net_salary

# Example usage:
employee = EmployeeSalaryCalculator(5000, 0.2)
print("Gross Salary:", employee.calculate_salary())
print("Net Salary:", employee.calculate_net_salary())
```

Output:

```
Year\AIAC\13.5.py'
Gross Salary: 5000
Net Salary: 4000.0
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>
```

Observation:

The procedural code was changed into a class-based design by using object-oriented principles.Encapsulation was done by keeping salary and tax rate as attributes and using class methods to handle the calculations.The improved code is broken down into smaller parts, making it easier to reuse and simpler to keep up or add new features to.

Task Description #6 : (Refactoring for Performance Optimization)
Prompt:

Refactor the following Python code to improve performance and using built-in functions or mathematical formulas.

Rename variables meaningfully and improve the code structure. Compare execution time between the legacy loop implementation and the optimized version.

Take input from the user for the limit.

Legacy Code:

```
total = 0
for i in range(1, 1000000):
    if i % 2 == 0:
        total += i
print(total)
```

Code:

```python
import time
def sum_of_even_numbers(limit):
    """Calculate the sum of even numbers up to a given limit.
    Args:
        limit (int): The upper limit for calculating the sum of even numbers.
        Returns:
        int: The sum of even numbers up to the given limit.
        Time Complexity: O(n), where n is the value of the limit.
        """
    #calculate the sum of even numbers up to the limit
    n=limit // 2
    return n * (n + 1)
#Take input from the user
try:
    limit = int(input("Enter the upper limit: "))
    start_time = time.time()
    result = sum_of_even_numbers(limit)
    end_time = time.time()
    print(f"The sum of even numbers up to {limit} is: {result}")
    print(f"Execution time: {end_time - start_time} seconds")
except ValueError:
    print("Please enter a valid integer.")
```

Output:

```
Year\AIAC\13.5.py'
Enter the upper limit: 23
The sum of even numbers up to 23 is: 132
Execution time: 6.67572021484375e-06 seconds
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>  c:; cd 'c:\Us
n.exe' 'c:\Users\Dhanush Reddy\.vscode\extensions\ms-py
Year\AIAC\13.5.py'
Enter the upper limit: 21
The sum of even numbers up to 21 is: 110
Execution time: 7.62939453125e-06 seconds
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>
```

Observation:

The refactored code boosts performance by swapping out the loop for a simple math formula and using built-in functions.This cuts down on the time it takes and gets rid of unneeded conditional checks inside the loop.Because of this, the improved version runs faster and gives the same results with code that's easier to read and maintain.

Task Description #7 : (Removing Hidden Side Effects)
Prompt:

Refactor the given Python code to remove hidden side effects caused by modifying a global mutable list.

Rewrite the function in a functional style so it returns a new list instead of mutating global state.

Add clear comments and Take input from the user.

Legacy Code:

```
data = []

def add_item(x):

    data.append(x)

add_item(10)

add_item(20)

print(data)
```

Code:

```
C:\Users\Dhanush Reddy\3RD Year\AIAC\13.5.py > 13.5.py > ...
def add_item_time_to_list(item, data_list):
    """Add an item and its time to a list.
    Args:
        item (str): The item to add.
        data_list (list): The list to add the item and time to.
    Returns:
        list: The updated list with the item and time added.
    """
    #Create a new list by concatenating the item and the current time
    return data_list + [item]

#Take input from user for the item to be added
try:
    item = input("Enter the item to add: ")
    #Initialize an empty list to store the items and times
    data_list = []
    #Call the function to add the item and time to the list
    updated_list = add_item_time_to_list(item, data_list)
    #Print the updated list
    print("Updated List:", updated_list)
except Exception as e:
    print("An error occurred:", e)
```

Output:

```
Enter the item to add: 5
Updated List: ['5']
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>
n.exe' 'c:\Users\Dhanush Reddy\.vscode\ext
Year\AIAC\13.5.py'
Enter the item to add: 7
Updated List: ['7']
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>
```

Observation:

The rewritten code gets rid of hidden side effects by not changing the global list.The function now works in a functional programming style by creating and returning a new list instead of changing shared mutable state.This makes the code more predictable and easier to maintain, and it also simplifies testing.

Task Description #8 : (Refactoring Complex Input Validation Logic)

Prompt:

Refactor the following Python password validation code to improve readability.

Create separate functions: check_length(password), check_digit(password), and check_uppercase(password) that return True/False.

Implement a validate_password(password) function that calls these functions and prints appropriate error messages if a rule fails.

Add comments and Take input from the user.

Legacy Code:

```
password = input("Enter password: ")
if len(password) >= 8:
    if any(c.isdigit() for c in password):
        if any(c.isupper() for c in password):
            print("Valid Password")
        else:
            print("Must contain uppercase")
    else:
        print("Must contain digit")
else:
print("Password too short")
```

Code:

C: > Users > Dhanush Reddy > 3RD Year > AIAC > 13.5.py > ...

```python
Expected Outcome:
o Separate validation functions with clear responsibility
"""
def check_length(password):
    """Check if the password is at least 8 characters long."""
    return len(password) >= 8

def check_digit(password):
    """Check if the password contains at least one digit."""
    return any(char.isdigit() for char in password)

def check_uppercase(password):
    """Check if the password contains at least one uppercase letter."""
    return any(char.isupper() for char in password)

def validate_password(password):
    """Validate the password by checking length, digit, and uppercase requirements."""
    if not check_length(password):
        print("Error: Password must be at least 8 characters long.")
        return False
    if not check_digit(password):
        print("Error: Password must contain at least one digit.")
        return False
    if not check_uppercase(password):
        print("Error: Password must contain at least one uppercase letter.")
        return False
    print("Password is valid.")
    return True

# Take password input from the user
user_password = input("Enter your password: ")
validate_password(user_password)
```

Output:

```
ear\AIAC\13.5.py'
Enter your password: 12345
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>  c:; cd 'c:\Users\Dhanus
n.exe' 'c:\Users\Dhanush Reddy\.vscode\extensions\ms-python.debug
ear\AIAC\13.5.py'
Enter your password: 12345678
Error: Password must contain at least one uppercase letter.
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>  c:; cd 'c:\Users\Dhanus
n.exe' 'c:\Users\Dhanush Reddy\.vscode\extensions\ms-python.debug
Year\AIAC\13.5.py'
Enter your password: Dhanush123
Password is valid.
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>
```

Observation:

The code is easier to read. Each validation rule is in its function. This makes the program simple to test and debug and maintain. It is better, than using a lot of conditional statements. The code is easy to understand. We can reuse the validation logic. The validation logic is still the same as it was before. The code

is easier to understand and reuse. The refactored code and the original code have the validation logic but the refactored code is simpler.